



JAIN
DEEMED-TO-BE UNIVERSITY

SCHOOL OF
COMPUTER
SCIENCE AND IT

DEPARTMENT OF CS & IT

BCA – ITH – 4TH SEMESTER

ACADEMIC YEAR : 2024 – 2025 (EVEN SEMESTER)

BATCH : 2023 – 2026

TITLE	PYTHON PROGRAMMING
SUBJECT CODE	23BCA4C03
HOURS PER WEEK	3 Hrs/Week
CREDITS	3

MODULE 2

TOPICS TO BE COVERED

DATA STRUCTURES

1. **Lists:** list operations
2. List Slicing
3. List methods, cloning lists, Indexing
4. List loop- mutability
5. List Aliasing
6. List Parameters
7. List Concatenation
8. Other operations on Sequence data type;
9. **Tuples**
10. Tuple assignment
11. Tuple as return value;
12. **Dictionaries:** operations and methods
13. Advanced list processing, List comprehension;
14. **Sets:** Creating a Set
15. Adding items to the set-
16. Removing item from the set
17. Operations on Set

DATA STRUCTURES

1. LISTS: LIST OPERATIONS

Python Lists

In Python, a **list** is a *built-in dynamic sized array (automatically grows and shrinks)*. We can store **all types of items** (including another list) in a list. A list may contain mixed type

of items, this is possible because a list mainly stores references at contiguous locations and actual items may be stored at different locations.

- List can contain duplicate items.
- List in Python are Mutable. Hence, we can modify, replace or delete the items.
- List are ordered. It maintains the order of elements based on how they are added.
- Accessing items in List can be done directly using their position (index), starting from 0.

```
a = [10, 20, 15]
print(a[0]) # access first item
a.append(11) # add item
a.remove(20) # remove item
print(a)
```

Output

```
10
[10, 15, 11]
```

LIST OPERATIONS

- Creating a List
- Accessing List Items
- Adding Items into List
- Updating Items into List
- Removing Items from List
- Iterating Over Lists
- Nested Lists

Creating a List

Here are some common methods to create a list:

Using Square Brackets

```
# List of integers
a = [1, 2, 3, 4, 5]
# List of strings
b = ['apple', 'banana', 'cherry']
# Mixed data types
c = [1, 'hello', 3.14, True]
print(a)
print(b)
print(c)
```

Output

```
[1, 2, 3, 4, 5]
```

```
['apple', 'banana', 'cherry']  
[1, 'hello', 3.14, True]
```

Using list() Constructor

We can also create a list by passing an **iterable** (like a [string](#), [tuple](#), or another **list**) to [list\(\)](#) function.

```
# From a tuple  
a = list((1, 2, 3, 'apple', 4.5))  
print(a)
```

Output

```
[1, 2, 3, 'apple', 4.5]
```

Creating List with Repeated Elements

We can create a list with repeated elements using the multiplication operator.

```
# Create a list [2, 2, 2, 2, 2]  
a = [2] * 5  
# Create a list [0, 0, 0, 0, 0, 0, 0]  
b = [0] * 7  
print(a)  
print(b)
```

Output

```
[2, 2, 2, 2, 2]  
[0, 0, 0, 0, 0, 0, 0]
```

Accessing List Elements

Elements in a list can be accessed using **indexing**. Python indexes start at **0**, so **a[0]** will access the first element, while **negative indexing** allows us to access elements from the end of the list. Like index -1 represents the last elements of list.

```
a = [10, 20, 30, 40, 50]  
# Access first element  
print(a[0])  
# Access last element  
print(a[-1])
```

Output

```
10  
50
```

Adding Elements into List

We can add elements to a list using the following methods:

- [append\(\)](#): Adds an element at the end of the list.
- [extend\(\)](#): Adds multiple elements to the end of the list.
- [insert\(\)](#): Adds an element at a specific position.

```
# Initialize an empty list
```

```
a = []  
# Adding 10 to end of list  
a.append(10)  
print("After append(10):", a)  
# Inserting 5 at index 0  
a.insert(0, 5)  
print("After insert(0, 5):", a)  
# Adding multiple elements [15, 20, 25] at the end  
a.extend([15, 20, 25])  
print("After extend([15, 20, 25]):", a)
```

Output

```
After append(10): [10]  
After insert(0, 5): [5, 10]  
After extend([15, 20, 25]): [5, 10, 15, 20, 25]
```

Updating Elements into List

We can change the value of an element by accessing it using its index.

```
a = [10, 20, 30, 40, 50]  
# Change the second element  
a[1] = 25  
print(a)
```

Output

```
[10, 25, 30, 40, 50]
```

Removing Elements from List

We can remove elements from a list using:

- [remove\(\)](#): Removes the first occurrence of an element.
- [pop\(\)](#): Removes the element at a specific index or the last element if no index is specified.
- [del statement](#): Deletes an element at a specified index.

```
a = [10, 20, 30, 40, 50]  
# Removes the first occurrence of 30  
a.remove(30)  
print("After remove(30):", a)  
# Removes the element at index 1 (20)  
popped_val = a.pop(1)  
print("Popped element:", popped_val)  
print("After pop(1):", a)  
# Deletes the first element (10)  
del a[0]  
print("After del a[0]:", a)
```

Output

```
After remove(30): [10, 20, 40, 50]
Popped element: 20
After pop(1): [10, 40, 50]
After del a[0]: [40, 50]
```

Iterating Over Lists

We can iterate the Lists easily by using a [for loop](#) or other iteration methods. Iterating over lists is useful when we want to do some operation on each item or access specific items based on certain conditions. Let's take an example to iterate over the list using **for** loop.

Using for Loop

```
a = ['apple', 'banana', 'cherry']
# Iterating over the list
for i in a:
    print(i)
```

Output

```
apple
banana
cherry
```

To learn various other methods, please refer to [iterating over lists](#).

Nested Lists in Python

A nested list is a list within another list, which is useful for representing matrices or tables. We can access nested elements by chaining indexes.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
# Access element at row 2, column 3
print(matrix[1][2])
```

Output

```
6
```

Basic Example on Python List

- [Length of a List](#)
- [Maximum in a List](#)
- [Swap two items in a list](#)
- [Check if an Element Exists](#)
- [Index of an Item](#)
- [Remove Duplicates from a List](#)
- [Reverse a List in Python](#)
- [Sum of a List](#)

- [Concatenate Two Lists](#)
- [Sort a List in Python](#)
- [Check for Sublist](#)

2. LIST SLICING

Python **list slicing** is fundamental concept that let us easily access specific elements in a list. We'll learn the syntax and how to use both positive and negative indexing for slicing with examples.

Example: Get the items from a list starting at position 1 and ending at position 4 (exclusive).

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get elements from index 1 to 4 (excluded)
print(a[1:4])
```

Output

```
[2, 3, 4]
```

Python List Slicing Syntax

```
list_name[start : end : step]
```

Parameters:

- **start (optional):** Index to begin the slice (inclusive). Defaults to 0 if omitted.
- **end (optional):** Index to end the slice (exclusive). Defaults to the length of list if omitted.
- **step (optional):** Step size, specifying the interval between elements. Defaults to 1 if omitted

List Slicing Examples

Let's see how to use list slicing in Python with the examples below.

Get all the items from a list

To retrieve all items from a list, we can use slicing without specifying any parameters.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
# Get all elements in the list
print(a[:])
print(a[:])
```

Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Explanation: Using `[:]` & `[:]` without specifying any **start**, **end**, or **step** returns all elements of the list.

`[:]` Copies the entire sequence

`[:]`

Copies the sequence with step control (default step = 1)

Get all items before/after a specific position

- To get all the items from a specific position to the **end** of the list, we can specify the **start** index and leave the end blank.
- And to get all the items before a specific index, we can specify the **end** index while leaving **start** blank.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Get elements starting from index 2 to the end of the list
```

```
b = a[2:]
```

```
print(b)
```

```
# Get elements starting from index 0 to index 3 (excluding 3th index)
```

```
c = a[:3]
```

```
print(c)
```

Output

```
[3, 4, 5, 6, 7, 8, 9]
```

```
[1, 2, 3]
```

Get all items between two positions

To extract elements between two specific positions, specify both the start and end indices

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Get elements from index 1
```

```
# to index 4 (excluding index 4)
```

```
b = a[1:4]
```

```
print(b)
```

Output

```
[2, 3, 4]
```

Get items at specified intervals

To extract elements at specific intervals, use the **step** parameter.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
# Get every second element from the list
```

```
# starting from the beginning
```

```
b = a[::2]
```

```
print(b)
```

```
# Get every third element from the list
```

```
# starting from index 2 to 8(exclusive)
```

```
c = a[1:8:3]
```

```
print(c)
```

Output

```
[1, 3, 5, 7, 9]
```

```
[2, 5, 8]
```

Out-of-bound slicing

In Python, **list slicing allows out-of-bound indexing without raising errors**. If we specify indices beyond the list length then it will simply return the available items.

Example: The slice `a[7:15]` starts at index 7 and attempts to reach index 15, but since the list ends at index 8, so it will return only the available elements (i.e. `[8,9]`).

Negative Indexing

Negative indexing is useful for **accessing elements from the end of the list**. The last element has an index of `-1`, the second last element `-2`, and so on.

Extract elements using negative indices

This example shows how to use negative numbers to access elements from the list starting from the end. **Negative indexing** makes it easy to get items without needing to know the exact length of the list.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]  
# Get elements starting from index -2  
# to end of list  
b = a[-2:]  
print(b)  
# Get elements starting from index 0  
# to index -3 (excluding 3th last index)  
c = a[:-3]  
print(c)  
# Get elements from index -4  
# to -1 (excluding index -1)  
d = a[-4:-1]  
print(d)  
# Get every 2nd elements from index -8  
# to -1 (excluding index -1)  
e = a[-8:-1:2]  
print(e)
```

Output

```
[8, 9]  
[1, 2, 3, 4, 5, 6]  
[6, 7, 8]  
[2, 4, 6, 8]
```

Reverse a list using slicing

We'll **reverse the entire list using a slicing** trick. By using a negative step value, we can move through the list in reverse order.

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9]  
# Get the entire list using negative step  
b = a[::-1]
```

```
print(b)
```

Output

```
[9, 8, 7, 6, 5, 4, 3, 2, 1]
```

Explanation: The negative **step (-1)** indicates that Python should traverse the list in reverse order, starting from the end. The slice **a[::-1]** starts from the end of the list and moves to the beginning which result in reversing list. It's a quick and easy way to get the list in reverse without changing the original list.

3. LIST METHODS, CLONING, INDEXING

[Python list](#) methods are built-in functions that allow us to perform various operations on **lists**, such as **adding**, **removing**, or **modifying** elements.

List Methods

Let's look at different list methods in Python:

- [append\(\)](#): Adds an element to the end of the list.
- [copy\(\)](#): Returns a shallow copy of the list.
- [clear\(\)](#): Removes all elements from the list.
- [count\(\)](#): Returns the number of times a specified element appears in the list.
- [extend\(\)](#): Adds elements from another list to the end of the current list.
- [index\(\)](#): Returns the index of the first occurrence of a specified element.
- [insert\(\)](#): Inserts an element at a specified position.
- [pop\(\)](#): Removes and returns the element at the specified position (or the last element if no index is specified).
- [remove\(\)](#): Removes the first occurrence of a specified element.
- [reverse\(\)](#): Reverses the order of the elements in the list.
- [sort\(\)](#): Sorts the list in ascending order (by default).

Examples of List Methods

append():

Syntax: *list_name.append(element)*

In the code below, we will add an element to the list.

```
a = [1, 2, 3]
# Add 4 to the end of the list
a.append(4)
print(a)
```

Output

```
[1, 2, 3, 4]
```

copy() / cloning a list :**Syntax:** *list_name.copy()*

In the code below, we will create a copy of a list.

```
a = [1, 2, 3]  
# Create a copy of the list  
b = a.copy()  
print(b)
```

Output

```
[1, 2, 3]
```

Using copy.deepcopy() for Deep Copy

For nested lists, a deep copy is required to ensure that changes to the cloned list do not affect the original. The copy module provides the [deepcopy\(\)](#) method for this purpose.

```
import copy  
a = [[1, 2], [3, 4], [5, 6]]  
# Copying the list using deepcopy  
b = copy.deepcopy(a)  
print(b)
```

Output

```
[[1, 2], [3, 4], [5, 6]]
```

Explanation:

- `deepcopy()` function recursively copies all objects inside the list.
- It ensures that changes to nested objects in the clone do not affect the original.

clear():**Syntax:** *list_name.clear()*

In the code below, we will clear all elements from the list.

```
a = [1, 2, 3]  
# Remove all elements from the list  
a.clear()  
print(a)
```

Output

```
[]
```

count():**Syntax:** *list_name.count(element)*

In the code below, we will count the occurrences of a specific element in the list.

```
a = [1, 2, 3, 2]
```

```
# Count occurrences of 2 in the list  
print(a.count(2))
```

Output

```
2
```

extend():

Syntax: *list_name.extend(iterable)*

In the code below, we will extend the list by adding elements from another list.

```
a = [1, 2]  
# Extend list a by adding elements from list [3, 4]  
a.extend([3, 4])  
print(a)
```

Output

```
[1, 2, 3, 4]
```

index():

Syntax: *list_name.index(element)*

In the code below, we will find the index of a specific element in the list.

```
a = [1, 2, 3]  
# Find the index of 2 in the list  
print(a.index(2))
```

Output

```
1
```

insert():

Syntax: *list_name.insert(index, element)*

In the code below, we will insert an element at a specific position in the list.

```
a = [1, 3]  
# Insert 2 at index 1  
a.insert(1, 2)  
print(a)
```

Output

```
[1, 2, 3]
```

pop():

Syntax: *list_name.pop(index)*

In the code below, we will remove the last element from the list.

```
a = [1, 2, 3]  
# Remove and return the last element in the list  
a.pop()  
print(a)
```

Output

```
[1, 2]
```

remove():**Syntax:** *list_name.remove(element)*

In the code below, we will remove the first occurrence of a specified element from the list.

```
a = [1, 2, 3]
# Remove the first occurrence of 2
a.remove(2)
print(a)
```

Output

```
[1, 3]
```

reverse():**Syntax:** *list_name.reverse()*

In the code below, we will reverse the order of the elements in the list.

```
a = [1, 2, 3]
# Reverse the list order
a.reverse()
print(a)
```

Output

```
[3, 2, 1]
```

sort():**Syntax:** *list_name.sort(key=None, reverse=False)*

In the code below, we will sort the elements of the list in ascending order

```
a = [3, 1, 2]
# Sort the list in ascending order
a.sort()
print(a)
```

Output

```
[1, 2, 3]
```

4. LIST LOOPS

Python provides several ways to iterate over list. The simplest and the most common way to iterate over a list is to use a **for loop**. This method allows us to access each element in the list directly.

Example: Print all elements in the list one by one using [for loop](#).

```
a = [1, 3, 5, 7, 9]
# On each iteration val represents the current item/element
for val in a:
    print(val)
```

Output

```
1
3
5
7
9
```

Let's see other different ways to iterate over a list in Python.

- *Using For-Loop and Range() Method*
- *Using While-Loop*
- *Using Enumerate()*
- *Using List Comprehension*
- *Using Iter and Next Functions*

Using for Loop with range()

We can use the [range\(\)](#) method with [for loop](#) to traverse the list. This method allow us to **access elements by their index**, which is useful if we need to know the position of an element or modify the list in place.

```
a = [1, 3, 5, 7, 9]
# Calculate the length of the list
n = len(a)
# Iterates over the indices from 0 to n-1 (i.e., 0 to 4)
for i in range(n):
    print(a[i])
```

Output

```
1
3
5
7
9
```

Using while Loop

This method is similar to the above method. Here we are using a [while loop](#) to iterate through a list. We first need to find the length of list using **len()**, then start at index 0 and access each item by its index then incrementing the index by 1 after each iteration.

```
a = [1, 3, 5, 7, 9]
# Start from the first index
i = 0
# The loop runs till the last index (i.e., 4)
while i < len(a):
    print(a[i])
    i += 1
```

Output

```
1
3
5
7
9
```

Using enumerate()

We can also use the [enumerate\(\)](#) function to iterate through the list. This method provides both the **index (i)** and the **value (val)** of each element during the loop.

```
a = [1, 3, 5, 7, 9]
# Here, i and val represents index and value respectively
for i, val in enumerate(a):
    print (i, val)
```

Output

```
0 1
1 3
2 5
3 7
4 9
```

Using List Comprehension

[List comprehension](#) is similar to **for loop**. It provides the shortest syntax for looping through list.

```
a = [1, 3, 5, 7, 9]
# On each iteration val is passed to print function
# And printed in the console.
[print(val) for val in a]
```

Output

```
1
3
5
7
9
```

5. LIST ALIASING

List aliasing is a method to **give a second name to an existing list**. Firstly, we will declare a list. Then, we declare another variable and assign the original list to the new variable. It will copy the list explicitly and have the same reference ID, i.e., it has the same memory address. We can check the id using the `id()` function. If we try to change any element in any of the lists, it will change in both lists.

Let's implement the list aliasing in Python.

Example Code:

```
original_list = [7, 8, 15, 99, 44]
print("Original List:", original_list, id(original_list))
alias_list = original_list
```

```
print("List after aliasing:", alias_list, id(alias_list))
```

Output:

```
Original List: [7, 8, 15, 99, 44] 2568590629632
```

```
List after aliasing: [7, 8, 15, 99, 44] 2568590629632
```

We declared a list of numbers (**original_list**) and printed the list with its memory address. Then, we declared another variable (**alias_list**) and assigned the original list to it. The list name **alias_list** is the second name for the **original_list**. Then, printed the aliased list and its memory address. It is seen that both the list has the same data and the memory address. If any changes are made to the **original_list**, it will also change the **alias_list**.

6. LIST PARAMETERS

When we pass a [list](#) as an argument to a function in [Python](#), it allows the function to access, process, and modify the elements of the list. In this article, we will see How to pass a list as an argument in Python, along with different use cases.

We can directly pass a list as a function argument.

```
def print_list(a):
    for item in a:
        print(item)
# Define a list of numbers
a = [1, 2, 3, 4, 5, 6]
print_list(a)
```

Output

```
1
2
3
4
5
6
```

Modifying the Original List

When a list is passed to a [function](#), the function can modify the original list since lists are mutable.

```
def add_item(items, new_item):
    items.append(new_item)
# Define a list 'a'
a = [1, 2, 3]
add_item(a, 4)
print(a)
```

Output

```
[1, 2, 3, 4]
```

Passing a Nested List

Python also allows passing a [nested list](#) (a list within a list) as a function argument.

```
def nested_list(nested_list):
    for sublist in nested_list:
        for item in sublist:
            print(item)
# Define a nested list
a = [[1, 2], [3, 4], [5, 6]]
nested_list(a)
```

Output

```
1
2
3
4
5
6
```

Using *args for Unpacking

We can use the *args syntax to unpack the elements of a list and pass them as separate arguments to a function:

```
def print_items(*args):
    for item in args:
        print(item)
# Define a list 'a'
a = [1, 2, 3]
print_items(*a)
```

Output

```
1
2
3
```

7. LIST CONCATENATION

Python provides several approaches to merge two lists. In this article, we will explore different methods to merge lists with their use cases. The simplest way to merge two lists is by using the **+** operator.

Let's take an example to merge two lists using [+ operator](#).

```
a = [1, 2, 3]
b = [4, 5, 6]
# Merge the two lists and assign
# the result to a new list
c = a + b
print(c)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation: The + operator creates a new list by concatenating **a** and **b**. This operation does not modify the original lists but returns a new combined list.

Lets see other different methods to merge two lists:

- Using the `extend()`
- Using the * Operator
- Using for loop
- Using list comprehension
- Using the `itertools.chain()`
- Which method to choose?

Using the `extend()`

Another common method is to use the `extend()` function, which modifies the original list by adding elements from another list.

```
a = [1, 2, 3]
b = [4, 5, 6]
# Add all elements from list 'b' to the end of list 'a'
a.extend(b)
print(a)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation: The `extend()` is an in-place operation that appends each element of list **b** to list **a**. This means that the original list **a** gets updated without creating a new list, which makes it more memory efficient for large lists.

Using the * Operator

We can use the `* operator` to unpack the elements of multiple lists and combine them into a new list.

```
a = [1, 2, 3]
b = [4, 5, 6]
# Use the unpacking operator to merge the lists
c = [*a, *b]
print(c)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation: The `* operator` unpacks the elements of **a** and **b**, placing them directly into the new list **c**.

Using for loop

We can also merge two lists using a simple `for loop`. This approach is provides more control if we need to perform additional operations on the elements while merging.

```
a = [1, 2, 3]
```

```
b = [4, 5, 6]
# Initialize an empty list to store the merged elements
res = []
# Append all elements from the first list
for val in a:
    res.append(val)
# Append all elements from the second list
for val in b:
    res.append(val)
print(res)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation:

- **for item in a:** Iterates over each element in list **a** and appends it to list **res**.
- **for item in b:** Iterates over each element in list **b** and appends it to list **res**.

Using list comprehension

[List comprehension](#) is an efficient and concise alternative to the **for loop** for merging lists. It allows us to iterate through each list and merge them into a new one in a single line of code.

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation: List comprehension iterates over both lists **a** and **b**, creating a new list by adding all items from each list.

Using the `itertools.chain()`

The [itertools.chain\(\)](#) method from the **itertools** module provides an efficient way to merge multiple lists. It doesn't create a new list but returns an **iterator**, which saves memory, especially with large lists.

```
# imports chain function from itertools module
from itertools import chain
a = [1, 2, 3]
b = [4, 5, 6]
# Use itertools.chain to merge the lists in an efficient manner
c = list(chain(a, b))
print(c)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

Explanation: `list(chain(a, b))` combines the lists **a** and **b**. The `chain(a, b)` creates an iterator that collect all items from **a** first then all items from **b**. Then, change this iterator into a list will gives us merged list.

Which method to choose?

- **+ Operator:** Quick and simple for small lists but can consume a lot of memory for larger ones.
- **extend() Method:** Efficiently merges large lists by modifying the original list in place.
- **Unpacking (*):** It provides a clean syntax, however, it may not be optimal for very large lists due to memory usage.
- **itertools.chain():** It works best for large lists. It merges without creating additional memory.
- **For Loop:** It provides more control to use complex merging conditions.
- **List Comprehension:** This is an alternative approach of using “for loop”, it has concise and cleaner syntax.

8. SEQUENCES IN PYTHON WITH TYPES AND EXAMPLES

Sequences in Python

Sequences are containers with items stored in a deterministic ordering. Each sequence data type comes with its unique capabilities.

There are many types of sequences in Python. Let's learn them with Examples.

Types of Sequences in Python

Python sequences are of **six types**, namely:

1. Strings
2. Lists
3. Tuples
4. Bytes Sequences
5. Bytes Arrays
6. `range()` objects

Strings in Python

In python, the string is a sequence of Unicode characters written inside a single or double-quote. Python does not have any **char** type as in other languages (C, C++), therefore, a single character inside the quotes will be of type **str** only.

1. To declare an **empty string**, use `str()` or it can be defined using empty string inside quotes.

Example of Empty String in Python

```
name = "PythonGeeks"  
print(name)
```

Output

```
PythonGeeks
```

2. Strings are **immutable** data types, therefore once declared, we can't alter the string. Though, we can reassign it to a new string.

Code

```
name = "PythonGeeks"  
print(name[6]) # outputs 'G'  
name[6] = 'g' # throws error  
print(name)
```

Output

```
G
```

Traceback (most recent call last):

```
File "/home/apoorve/Documents/PythonGeeks/python_sequences/main.py", line 3, in  
<module>
```

```
    name[7] = 'g'
```

```
TypeError: 'str' object does not support item assignment
```

Lists in Python

Lists are a single storage unit to store multiple data items together. It's a mutable data structure, therefore, once declared, it can still be altered.

A list can hold **strings, numbers, lists, tuples, dictionaries, etc.**

1. To declare a list, either use **list()** or square brackets [], containing comma-separated values.

Example of Lists in Python

```
list_1 = ["PythonGeeks", "Sequences", "Tutorial"] # [all string list]  
print(f'List 1: {list_1}')  
list_2 = list() # [empty list]  
print(f'List 2: {list_2}')  
list_3 = [2021, ['hello', 2020], 2.0] # [integer, list, float]  
print(f'List 3: {list_3}')  
list_4 = [{'language': 'Python'}, (1,2)] # [dictionary, tuple]  
print(f'List 4: {list_4}')
```

Output

```
List 1: ['PythonGeeks', 'Sequences', 'Tutorial']
```

```
List 2: []
```

```
List 3: [2021, ['hello', 2020], 2.0]
```

```
List 4: [{'language': 'Python'}, (1, 2)]
```

2. Let's check the mutability of lists, now.

Code

```
list_1 = ["PythonGeeks", "Sequences", "Tutorial"] # [all string list]
```

```
list_1[2] = "Blog"
```

```
print(list_1)
```

Output

```
['PythonGeeks', 'Sequences', 'Blog']
```

Tuples in Python

Just like Lists, Tuples can store multiple data items of different data types. The only difference is that they are **immutable** and are stored inside the parenthesis **()**.

1. To declare a tuple, either use **tuple()** or parenthesis, containing comma-separated values.

Example of Tuple in Python:

```
tuple_1 = ("PythonGeeks", "Sequences", "Tutorial") # [all string tuple]
```

```
print(f'tuple 1: {tuple_1}')
```

```
tuple_2 = tuple() # [empty tuple]
```

```
print(f'tuple 2: {tuple_2}')
```

```
tuple_3 = [2021, ('hello', 2020), 2.0] # [integer, tuple, float]
```

```
print(f'tuple 3: {tuple_3}')
```

```
tuple_4 = [{'language': 'Python'}, [1,2]] # [dictionary, list]
```

```
print(f'tuple 4: {tuple_4}')
```

Output:

```
tuple 1: ('PythonGeeks', 'Sequences', 'Tutorial')
```

```
tuple 2: ()
```

```
tuple 3: [2021, ('hello', 2020), 2.0]
```

```
tuple 4: [{'language': 'Python'}, [1, 2]]
```

2. Let's test the immutability of tuples now.

Code:

```
tuple_1 = ("PythonGeeks", "Sequences", "Tutorial") # [all string tuple]
```

```
tuple_1[2] = "Blog"
```

```
print(tuple_1)
```

Output:

```
Traceback (most recent call last):
```

```
File "/home/apoorve/Documents/PythonGeeks/python_sequences/main.py", line 2, in  
<module>  
tuple_1[3] = "Blog"  
TypeError: 'tuple' object does not support item assignment
```

Byte Sequences in Python

The **bytes()** function returns an **immutable** bytes sequence in between quotes, preceded by a 'B' or 'b'.

1. To declare an empty bytes object, use **bytes(size)**, where size is the number of empty bytes we want to generate.

Example of Byte Sequences in Python:

```
size = 10  
b = bytes(size)  
print(b)
```

Output:

```
b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

2. Alternatively, we can pass a list of numbers in bytes() function as follows:

Code:

```
b = bytes([5,8])  
print(b)
```

Output:

```
B'\x05\x08'
```

3. In the case of strings, we need to pass a second parameter as the type of encoding.

Code:

```
b = bytes("PythonGeeks", 'utf-8')  
print(b)
```

Output:

```
b'PythonGeeks'
```

Byte Arrays in Python

Just like bytes sequence, bytes arrays also return a bytes object. The only difference is that they are **mutable**.

1. Let's see how to use them.

Example of Byte Arrays in Python:

```
print(bytearray(10))  
print(bytearray([5,8]))  
print(bytearray("PythonGeeks", 'utf-8'))
```

Output:

```
bytearray(b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00')
```

```
bytearray(b'\x05\x08')  
bytearray(b'PythonGeeks')
```

2. Try **mutating a byte** in the array.

Code:

```
b = bytearray([10,20,60,40,50])  
print(f'Before: {b}')  
b[2] = 30  
print(f'Later: {b}')
```

Output:

```
Before: bytearray(b'\n\n\x14<(2')  
Later: bytearray(b'\n\n\x14\x1e(2')
```

range() object in Python

It returns a **sequence of integers** in the specified range. By default, it starts the sequence from 0 (if not specified).

Code:

```
sequence = range(10)  
print(sequence)
```

Output:

```
range(0,10)
```

Code:

```
for i in range(2,5):  
    print(i)
```

Output:

```
2  
3  
4
```

As a third parameter, we can specify the **increment count** (here, 2). By default it is set to 1.

Code:

```
sequence= range(1, 10, 2)  
for i in sequence:  
    print(i)
```

Output:

```
1  
3  
5  
7  
9
```

Operations on Sequences in Python

We now know the types of sequences, but now, it's time to see what all operations can we perform on them. Here, we will focus on the most commonly used operations.

1. Concatenation Operator in Python

(+) operator is used to join two sequences of the same type.

Code:

```
print(["Python"] + ["Geeks"])
```

Output:

```
["Python", "Geeks"]
```

2. Repeat Operator in Python

(*) operator repeats a sequence specified number of times.

Code:

```
print(("PythonGeeks", 1) * 2)
```

Output:

```
('PythonGeeks', 1, 'PythonGeeks', 1)
```

3. Membership Operator in Python

These are used to check whether a value is present in a sequence or not using the "in" and "not in" operators.

Code:

```
dict = {  
    "lang": "Python",  
    "platform": "PythonGeeks"  
}  
print("lang" in dict)  
print("code" not in dict)
```

Output:

```
True  
True
```

4. Slicing Operator in Python

[:] operator returns a part of the sequence between a given range.

Code:

```
list_1 = [3,2,5,6,7]  
print(list_1[:5])  
print(list_1[2:4])  
print(list_1[-1:])
```

Output:

```
[3, 2, 5, 6, 7]  
[5, 6]  
[7]
```

Python Sequence Functions and Methods

Now, it's time to discuss some important functions and methods used for sequences of all the types.

1. **len(sequence)** : Returns length of a sequence.
2. **index(index)**: Returns index of the first occurrence of an element in a sequence.
3. **min(sequence)**: Returns the minimum value of a sequence.
4. **max(sequence)**: Returns maximum value of a sequence.
5. **count()**: Returns the count of a number of occurrences of an element in a sequence.
6. **append(value)**: Adds the **value** at the end of the sequence.
7. **clear()**: Clears all the contents of the sequence.
8. **insert(value, index)**: Inserts the **value** at the index "**index**" of the sequence.
9. **pop(index)**: Returns and deletes elements at index "**index**". By default, the last element is deleted from the sequence.
10. **remove(value)**: Removes the first occurrence of **value** from the sequence.
11. **reverse()**: Reverse the sequence

Example of Sequence Function in Python

```
test = [2, 4, 6, 8, 10, 10]
print(len(test))
print(test.index(6))
print(min(test))
print(max(test))
print(test.count(10))
test.append(11)
print(test)
test.clear()
print(test)
test = [2, 4, 6, 8, 10, 10]
test.insert(9,4)
print(test)
print(test.pop())
test.remove(10)
print(test)
test.reverse()
print(test)
```

Output:

```
6
2
2
10
2
[2, 4, 6, 8, 10, 10, 11]
```

```

[]
[2, 4, 6, 8, 10, 10, 4]
4
[2, 4, 6, 8, 10]
[10, 8, 6, 4, 2]

```

Interview Questions on Sequences in Python

To make you understand some more important concepts of Python Sequences, we have curated a list of Top 5 Question and Answers that will help you hit the pot of gold in your upcoming interviews:

Q1. What will be the output of the code below?

```

b = range(0,10)
print(type(b))

```

Ans 1.

```
>>> <class 'range'>
```

Though printing **b** will return a list of numbers from 0 to 10, that doesn't mean it is of type list. **range()** object is of type **range**.

Q2. Is it possible to write a **for loop to create a list from space-separated input in one line? If yes, then how?**

Ans 2. Yes, it is possible to write a one-liner code for creating a list or performing any other operation on list elements. This is how we can do that.

```

list_1 = [i for i in input("Input: ").split()] # input 2 5 8 10
print(list_1)

```

Here, **input().split()** will take space-separated input for list elements and append them to the list **list_1**.

Output:

```

Input: 2 5 8 10
['2', '5', '8', '10']

```

Q3. **id() function in python, accepts a single parameter and is used to return the identity of an object. In simple words, objects pointing to the same memory address have the same id and vice-a-versa.**

Based on this, tell whether the id for **list1** and **list2** will be the same or different?

```

list1= ["PythonGeeks"]
list2 = list1

```

Ans 3. The id for both lists will be the same. This is because, **list2 = list1** doesn't mean that we are creating a copy of list1 in a different memory block, but it means that we are creating another list that points to the same memory location.

Therefore, they have the same id, and any change made in **list2** will be reflected on **list1** and vice-a-versa.

Q4. Write code to convert the following tuple of the list to a simple tuple.

Example: ([1,2],[5,6,7]) => (1,2,5,6,7)

Ans 4. Code is as follows:

```
tuple_before = ([2,3,4], [8,9],[20])
# Combination of sum() and tuple() function
tuple_after = tuple(sum(tuple_before, []))
print(tuple_after)
```

Output:

(2,3,4,8,9,20)

This is because the **sum()** function adds an empty list to each element of the tuple to create a list of the same numbers as in the input tuple. Then the list is converted into tuple using the **tuple()** function.

Q5. How to partition a string at a particular word, into a tuple of substrings?

Ans 5. Suppose our string is "PythonGeeks is the best platform for self-paced learning of Python".

Now if we want to split it at the word " best " into a tuple of substrings, here is how we do it using Python's built-in string method **partition()**

Code:

```
string = "PythonGeeks is the best platform for self-paced learning of Python"
print(string.partition(" best "))
```

Output:

('PythonGeeks is the', 'best ', 'platform for self-paced learning of Python')

9. TUPLES

Python Tuple is a collection of objects separated by commas. A tuple is similar to a Python list in terms of indexing, nested objects, and repetition but the main difference between both is Python tuple is immutable, unlike the Python list which is mutable.

```
# Note : In case of list, we use square
# brackets []. Here we use round brackets ()
t = (10, 20, 30)
print(t)
print(type(t))
```

Output

(10, 20, 30)

```
<class 'tuple'>
```

What is Immutable in Tuples?

Unlike [Python lists](#), tuples are immutable. Some Characteristics of Tuples in Python.

- Like Lists, tuples are ordered and we can access their elements using their index values
- We cannot update items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

Let us see this with an example.

```
t = (1, 2, 3, 4, 5)
# tuples are indexed
print(t[1])
print(t[4])
# tuples contain duplicate elements
t = (1, 2, 3, 4, 2, 3)
print(t)
# updating an element
t[1] = 100
print(t)
```

Output:

```
2
5
(1, 2, 3, 4, 2, 3)
Traceback (most recent call last):
  File "Solution.py", line 12, in <module>
    t[1] = 100
TypeError: 'tuple' object does not support item assignment
```

Accessing Values in Python Tuples

Tuples in Python provide two ways by which we can access the elements of a tuple.

Python Access Tuple using a Positive Index

Using square brackets we can get the values from tuples in Python.

```
t = (10, 5, 20)
print("Value in t[0] = ", t[0])
print("Value in t[1] = ", t[1])
print("Value in t[2] = ", t[2])
```

Output

```
Value in t[0] = 10
Value in t[1] = 5
Value in t[2] = 20
```

Access Tuple using Negative Index

In the above methods, we use the positive index to access the value in Python, and here we will use the negative index within [].

```
t = (10, 5, 20)
print("Value in t[-1] = ", t[-1])
print("Value in t[-2] = ", t[-2])
print("Value in t[-3] = ", t[-3])
```

Output

```
Value in t[-1] = 20
Value in t[-2] = 5
Value in t[-3] = 10
```

Different Operations Related to Tuples

Below are the different operations related to tuples in Python:

Traversing Items of Python Tuples

Like List Traversal, we can traverse through a tuple using for loop.

```
# Define a tuple
t = (1, 2, 3, 4, 5)
# Traverse through each item in the tuple
for x in t:
    print(x, end=" ")
```

Output

```
1 2 3 4 5
```

Concatenation of Python Tuples

To [Concatenation](#) of Python Tuples, we will use plus operators(+).

```
# Code for concatenating 2 tuples
t1 = (0, 1, 2, 3)
t2 = ('python', 'geek')
# Concatenating above two
print(t1 + t2)
```

Output

```
(0, 1, 2, 3, 'python', 'geek')
```

Nesting of Python Tuples

A nested tuple in Python means a tuple inside another tuple.

```
# Code for creating nested tuples
t1 = (0, 1, 2, 3)
t2 = ('python', 'geek')
t3 = (t1, t2)
print(t3)
```

Output

```
((0, 1, 2, 3), ('python', 'geek'))
```

Repetition Python Tuples

We can create a tuple of multiple same elements from a single element in that tuple.

Code to create a tuple with repetition

```
t = ('python,')*3
```

```
print(t)
```

Output

```
('python, 'python, 'python')
```

Try the above without a comma and check. You will get tuple3 as a string 'pythonpythonpython'.

Slicing Tuples in Python

Slicing a Python tuple means dividing a tuple into small tuples using the indexing method. In this example, we slice the tuple from index 1 to the last element. In the second print statement, we printed the tuple using reverse indexing. And in the third print statement, we printed the elements from index 2 to 4.

code to test slicing

```
t = (0 ,1, 2, 3)
```

```
print(t[1:])
```

```
print(t[::-1])
```

```
print(t[2:4])
```

Output

```
(1, 2, 3)
```

```
(3, 2, 1, 0)
```

```
(2, 3)
```

Note: In Python slicing, the end index provided is not included.

Deleting a Tuple in Python

In this example, we are deleting a tuple using '[del](#)' keyword. The output will be in the form of error because after deleting the tuple, it will give a NameError.

Note: Remove individual tuple elements is not possible, but we can delete the whole Tuple using Del keyword.

Code for deleting a tuple

```
t = ( 0, 1)
```

```
del t
```

```
print(t)
```

Output:

```
Hangup (SIGHUP)
```

```
Traceback (most recent call last):
```

```
File "Solution.py", line 5, in <module>
```

```
print(t)
```

NameError: name 't' is not defined

Finding the Length of a Python Tuple

To find the length of a tuple, we can use Python's `len()` function and pass the tuple as the parameter.

```
# Code for printing the length of a tuple
```

```
t = ('python', 'geek')
```

```
print(len(t))
```

Output

```
2
```

Multiple Data Types With Tuple

Tuples in Python are heterogeneous in nature. This means tuples support elements with multiple datatypes.

```
# tuple with different datatypes
```

```
t = ("immutable", True, 23)
```

```
print(t)
```

Output

```
('immutable', True, 23)
```

Converting a List to a Tuple

We can convert a [list](#) in Python to a tuple by using the `tuple()` constructor and passing the list as its parameters.

```
# Code for converting a list and a string into a tuple
```

```
a = [0, 1, 2]
```

```
t = tuple(a)
```

```
print(t)
```

Output

```
(0, 1, 2)
```

Output:

Tuples take a single parameter which may be a list, string, set, or even a dictionary (only keys are taken as elements), and converts them to a tuple.

Tuples in a Loop

We can also create a tuple with a single element in it using [loops](#).

```
# python code for creating tuples in a loop
```

```
t = ('gfg',)
```

```
# Number of time loop runs
```

```
n = 5
```

```
for i in range(int(n)):
```

```
    t = (t,)
```

```
    print(t)
```

Output

```
(('gfg',),)
(((('gfg',),),),)
((((('gfg',),),),),)
((((('gfg',),),),),)
((((('gfg',),),),),),)
```

Different Ways of Creating a Tuple

- Using round brackets
- Without Brackets
- Tuple Constructor
- Empty Tuple
- Single Element Tuple
- Using Tuple Packing

Using Round Brackets

```
t = ("gfg", "Python")
print(t)
```

Output

```
('gfg', 'Python')
```

Using Comma Separated

```
# Creating a tuple without brackets
t = 4, 5, 6
print(t) # Output: (4, 5, 6)
```

Output

```
(4, 5, 6)
```

Using Tuple Constructor

```
# Creating a tuple using the tuple() constructor
t = tuple([7, 8, 9])
print(t) # Output: (7, 8, 9)
```

Output

```
(7, 8, 9)
```

Creating an Empty Tuple

```
# Creating an empty tuple
t = ()
print(t) # Output: ()
```

Output

```
()
```

Single Element Tuple

```
# Creating a single-element tuple
t = (10, ) # Comma is important here
```

```

print(t) # Output: (10,)
print(type(t))
# What if we do not use comma
t = (10) # This an integer (not a tuple)
print(t)
print(type(t))

```

Output

```

(10,)
<class 'tuple'>
10
<class 'int'>

```

Tuple Packing

```

# Tuple packing
a, b, c = 11, 12, 13
t = (a, b, c)
print(t) # Output: (11, 12, 13)

```

Output

```

(11, 12, 13)

```

10. TUPLE ASSIGNMENT

Tuple assignment is a feature that allows you to assign multiple variables simultaneously by unpacking the values from a tuple (or other iterable) into those variables.

Tuple assignment is a concise and powerful way to assign values to multiple variables in a single line of code.

```

#Creating a tuple
my_tuple = (1, 2, 3)
# Tuple assignment
a, b, c = my_tuple
# Now, a is 1, b is 2, and c is 3

```

In this example, the values from the my_tuple tuple are unpacked and assigned to the variables a, b, and c in the same order as they appear in the tuple.

Tuple assignment is not limited to tuples; it can also work with other iterable types like lists:

```

# Creating a list
my_list = [4, 5, 6]
# Tuple assignment with a list
x, y, z = my_list
# Now, x is 4, y is 5, and z is 6

```

Tuple assignment can be used to swap the values of two variables without needing a

temporary variable:

```
# Swapping values using tuple assignment
```

```
a = 1
```

```
b = 2
```

```
a, b = b, a
```

```
# Now, a is 2, and b is 1
```

Tuple assignment is a versatile feature in Python and is often used when you want to work with multiple values at once, making your code more readable and concise.

Tuple Assignment Python Example

Example 1: Basic Tuple Assignment

```
# Creating a tuple
```

```
coordinates = (3, 4)
```

```
# Unpacking the tuple into two variables
```

```
x, y = coordinates
```

```
# Now, x is 3, and y is 4
```

Example 2: Multiple Variables Assigned at Once

```
# Creating a tuple
```

```
person_info = ("Alice", 30, "New York")
```

```
# Unpacking the tuple into multiple variables
```

```
name, age, city = person_info
```

```
# Now, name is "Alice," age is 30, and city is "New York"
```

Example 3: Swapping Values

```
# Swapping values of two variables using tuple assignment
```

```
a = 5
```

```
b = 10
```

```
a, b = b, a
```

```
# Now, a is 10, and b is 5
```

Example 4: Unpacking a Tuple Inside a Loop

```
# Creating a list of tuples
```

```
points = [(1, 2), (3, 4), (5, 6)]
```

```
# Iterating through the list and unpacking each tuple
```

```
for x, y in points:
```

```
    print(f"X-coordinate: {x}, Y-coordinate: {y}")
```

```
# Output:
```

```
# X-coordinate: 1, Y-coordinate: 2
```

```
# X-coordinate: 3, Y-coordinate: 4
```

```
# X-coordinate: 5, Y-coordinate: 6
```

Example 5: Ignoring Unwanted Values

```
# Creating a tuple
```

```
values = (10, 20, 30, 40, 50)
```

```
# Unpacking only the first and last values
```

```
first, _, last = values
```

```
# Now, first is 10, and last is 50, and other values are ignored
```

These examples demonstrate various uses of tuple assignment in Python, from basic variable assignment to more advanced scenarios like swapping values or ignoring unwanted elements in the tuple. Tuple assignment is a powerful tool for working with structured data in Python.

11. TUPLE AS RETURN VALUE

we can return multiple values from a function. Following are different ways

1) Using Object

This is similar to C/C++ and Java, we can create a class (in C, struct) to hold multiple values and return an object of the class.

```
# A Python program to return multiple
```

```
# values from a method using class
```

```
class Test:
```

```
    def __init__(self):
```

```
        self.str = "geeksforgeeks"
```

```
        self.x = 20
```

```
# This function returns an object of Test
```

```
def fun():
```

```
    return Test()
```

```
# Driver code to test above method
```

```
t = fun()
```

```
print(t.str)
```

```
print(t.x)
```

Output

```
geeksforgeeks
```

```
20
```

2) Using Tuple

A Tuple is a comma separated sequence of items. It is created with or without (). Tuples are immutable. See [this](#) for details of tuple and list.

```
# A Python program to return multiple
```

```
# values from a method using tuple
```

```
# This function returns a tuple
```

```
def fun():
```

```
    str = "geeksforgeeks"
```

```
    x = 20
```

```
    return str, x # Return tuple, we could also write (str, x)
```

```
# Driver code to test above method
```

```
str, x = fun() # Assign returned tuple
```

```
print(str)
```

```
print(x)
```

Output

```
geeksforgeeks
```

```
20
```

3) Using a list

A list is like an array of items created using square brackets. They are different from arrays as they can contain items of different types. Lists are different from tuples as they are mutable.

```
# A Python program to return multiple
```

```
# values from a method using list
```

```
# This function returns a list
```

```
def fun():
```

```
    str = "geeksforgeeks"
```

```
    x = 20
```

```
    return [str, x]
```

```
# Driver code to test above method
```

```
list = fun()
```

```
print(list)
```

Output

```
['geeksforgeeks', 20]
```

4) Using a Dictionary

A Dictionary is similar to hash or map in other languages.

```
# A Python program to return multiple
```

```
# values from a method using dictionary
```

```
# This function returns a dictionary
```

```
def fun():
```

```
    d = dict();
```

```
    d['str'] = "GeeksforGeeks"
```

```
    d['x'] = 20
```

```
    return d
```

```
# Driver code to test above method
```

```
d = fun()
```

```
print(d)
```

Output

```
{'x': 20, 'str': 'GeeksforGeeks'}
```

5) Using Data Class (Python 3.7+)

In Python 3.7 and above the Data Class can be used to return a class with automatically added unique methods. The Data Class module has a decorator and functions for automatically adding generated special methods such as `__init__()` and `__repr__()` in the user-defined classes.

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Book_list:
    name: str
    perunit_cost: float
    quantity_available: int = 0
    # function to calculate total cost
    def total_cost(self) -> float:
        return self.perunit_cost * self.quantity_available
book = Book_list("Introduction to programming.", 300, 3)
x = book.total_cost()
# print the total cost
# of the book
print(x)
# print book details
print(book)
# 900
Book_list(name='Python programming.',
          perunit_cost=200,
          quantity_available=3)
```

Output

```
900
Book_list(name='Introduction to programming.', perunit_cost=300,
quantity_available=3)
```

6. Using 'yield'

One alternative approach for returning multiple values from a function in Python is to use the yield keyword in a generator function. A generator function is a special type of function that returns an iterator object, which generates a sequence of values on the fly, one value at a time.

To return multiple values from a generator function, you can use the yield keyword to yield each value in turn. The generator function will then pause execution until the next value is requested, at which point it will resume execution and yield the next value. This process continues until the generator function completes execution or encounters a return statement.

Here is an example of how this can be done:

```
def get_values():
    yield 42
    yield 'hello'
    yield [1, 2, 3]
# Test code
```

```
result = get_values()
print(next(result)) # should print 42
print(next(result)) # should print 'hello'
print(next(result)) # should print [1, 2, 3]
```

Output

```
42
hello
[1, 2, 3]
```

Time complexity: $O(1)$ because it only performs a constant number of operations (yields) regardless of the size of the input.

Auxiliary space: $O(1)$ because it only stores a constant number of variables (yielded values) in memory at any given time.

12. DICTIONARIES: OPERATIONS AND METHODS

A **Python dictionary** is a data structure that stores the value in **key: value** pairs. Values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and must be *immutable*.

Example: Here, The data is stored in key : value pairs in dictionaries, which makes it easier to find values.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
```

How to Create a Dictionary

In [Python](#), a dictionary can be created by placing a sequence of elements within curly {} braces, separated by a 'comma'.

```
# create dictionary using { }
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d1)
# create dictionary using dict() constructor
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
print(d2)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
{'a': 'Geeks', 'b': 'for', 'c': 'Geeks'}
```

- From Python 3.7 Version onward, Python dictionary are Ordered.

- **Dictionary keys are case sensitive:** the same name but different cases of Key will be treated distinctly.
- **Keys must be immutable:** This means keys can be strings, numbers, or tuples but not lists.
- **Keys must be unique:** Duplicate keys are not allowed and any duplicate key will overwrite the previous value.
- Dictionary internally uses [Hashing](#). Hence, operations like search, insert, delete can be performed in **Constant Time**.
- [Accessing Dictionary Items](#)
- [Adding and Updating Dictionary Items](#)
- [Removing Dictionary Items](#)
- [Iterating Through a Dictionary](#)
- [Nested Dictionaries](#)

Accessing Dictionary Items

We can access a value from a dictionary by using the **key** within square brackets or [get\(\)](#) method.

```
d = { "name": "Alice", 1: "Python", (1, 2): [1,2,4] }
# Access using key
print(d["name"])
# Access using get()
print(d.get("name"))
d = { "name": "Alice", 1: "Python", (1, 2): [1,2,4] }
# Access using key
print(d["name"])
# Access using get()
print(d.get("name"))
```

Output

```
Alice
Alice
```

Adding and Updating Dictionary Items

We can add new key-value pairs or update existing keys by using assignment.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
# Adding a new key-value pair
d["age"] = 22
# Updating an existing value
d[1] = "Python dict"
print(d)
```

Output

```
{1: 'Python dict', 2: 'For', 3: 'Geeks', 'age': 22}
```

Removing Dictionary Items

We can remove items from dictionary using the following methods:

- `del`: Removes an item by key.
- `pop()`: Removes an item by key and returns its value.
- `clear()`: Empties the dictionary.
- `popitem()`: Removes and returns the last key-value pair.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks', 'age':22}
# Using del to remove an item
del d["age"]
print(d)
# Using pop() to remove an item and return the value
val = d.pop(1)
print(val)
# Using popitem to removes and returns
# the last key-value pair.
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")
# Clear all items from the dictionary
d.clear()
print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: 'Geeks'}
Geeks
Key: 3, Value: Geeks
{}

```

Iterating Through a Dictionary

We can iterate over **keys** [using [keys\(\) method](#)] , **values** [using [values\(\) method](#)] or both [using [item\(\) method](#)] with a [for loop](#).

```
d = {1: 'Geeks', 2: 'For', 'age':22}
# Iterate over keys
for key in d:
    print(key)
# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

Output

```
1
2
age
Geeks
For
22
1: Geeks
2: For
age: 22
```

Nested Dictionaries

Example of Nested Dictionary:

```
d = {1: 'Geeks', 2: 'For',
      3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
print(d)
```

Output

```
{1: 'Geeks', 2: 'For', 3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}
```

Python Dictionary Problems

[Length of a Dictionary](#)

[Check if a Key Exists](#)

[Access a Value by Key](#)

[Remove a Key from a Dictionary](#)

[Remove keys with substring values](#)

[Sum All Numeric Values in a Dictionary](#)

[Find Keys with Maximum Value](#)

[Remove Duplicates from a Dictionary](#)

[Filter Dictionary by Key Prefix](#)

[Count Frequency of Elements Using Dictionary](#)

[Check if two arrays are equal or not](#)

[Max distance between two occurrences in array](#)

[2 Sum – Count Pairs with target sum](#)

[3 Sum – Count all triplets with target sum](#)

[Count all pairs with absolute difference equal to k](#)

[Remove minimum elements such that no common elements exist in two arrays](#)

[Check If Array Pair Sums Divisible by k](#)

[Longest subarray with sum divisible by K](#)

[Longest Subarray having Majority Elements Greater Than K](#)

[Count distinct elements in every window of size k](#)

Python Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
<code>clear()</code>	Removes all the elements from the dictionary
<code>copy()</code>	Returns a copy of the dictionary
<code>fromkeys()</code>	Returns a dictionary with the specified keys and value
<code>get()</code>	Returns the value of the specified key
<code>items()</code>	Returns a list containing a tuple for each key value pair
<code>keys()</code>	Returns a list containing the dictionary's keys
<code>pop()</code>	Removes the element with the specified key
<code>popitem()</code>	Removes the last inserted key-value pair
<code>setdefault()</code>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<code>update()</code>	Updates the dictionary with the specified key-value pairs
<code>values()</code>	Returns a list of all the values in the dictionary

Python Dictionary `clear()` Method

Example

Remove all elements from the `car` list:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
car.clear()
print(car)
```

Definition and Usage

The `clear()` method removes all the elements from a dictionary.

Syntax

```
dictionary.clear()
```

`copy()` Method

Example

Copy the `car` dictionary:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = car.copy()
print(x)
```

Definition and Usage

The `copy()` method returns a copy of the specified dictionary.

Syntax

```
dictionary.copy()
```

fromkeys() Method

Example

Create a dictionary with 3 keys, all with the value 0:

```
x = ('key1', 'key2', 'key3')
y = 0
thisdict = dict.fromkeys(x, y)
print(thisdict)
```

Definition and Usage

The `fromkeys()` method returns a dictionary with the specified keys and the specified value.

Syntax

```
dict.fromkeys(keys, value)
```

Parameter Values

Parameter	Description
<i>keys</i>	Required. An iterable specifying the keys of the new dictionary
<i>value</i>	Optional. The value for all keys. Default value is None

Example

Same example as above, but without specifying the value:

```
x = ('key1', 'key2', 'key3')
thisdict = dict.fromkeys(x)
print(thisdict)
```

get() Method

Example

Get the value of the "model" item:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
```

```
}  
x = car.get("model")  
print(x)
```

Definition and Usage

The `get()` method returns the value of the item with the specified key.

Syntax

`dictionary.get(keyname, value)`

Parameter Values

Parameter	Description
<i>keyname</i>	Required. The keyname of the item you want to return the value from
<i>value</i>	Optional. A value to return if the specified key does not exist. Default value None

Example

Try to return the value of an item that do not exist:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.get("price", 15000)  
print(x)
```

items() Method

Example

Return the dictionary's key-value pairs:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.items()  
print(x)
```

Definition and Usage

The `items()` method returns a view object. The view object contains the key-value pairs of the dictionary, as tuples in a list.

The view object will reflect any changes done to the dictionary, see example below.

Syntax

dictionary.items()**Example**

When an item in the dictionary changes value, the view object also gets updated:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.items()  
car["year"] = 2018  
print(x)
```

keys() Method**Example**

Return the keys:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.keys()  
print(x)
```

Definition and Usage

The **keys()** method returns a view object. The view object contains the keys of the dictionary, as a list.

The view object will reflect any changes done to the dictionary, see example below.

Syntax***dictionary.keys()*****Example**

When an item is added in the dictionary, the view object also gets updated:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.keys()  
car["color"] = "white"  
print(x)
```

pop() Method**Example**

Remove "model" from the dictionary:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
car.pop("model")  
print(car)
```

Definition and Usage

The `pop()` method removes the specified item from the dictionary.

The value of the removed item is the return value of the `pop()` method, see example below.

Syntax

`dictionary.pop(keyname, defaultvalue)`

Parameter Values

Example

Parameter	Description
<i>keyname</i>	Required. The keyname of the item you want to remove
<i>defaultvalue</i>	Optional. A value to return if the specified key do not exist. If this parameter is not specified, and the no item with the specified key is found, an error is raised

The value of the removed item is the return value of the `pop()` method:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.pop("model")  
print(x)
```

popitem() Method

Example

Remove the last item from the dictionary:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
car.popitem()  
print(car)
```

Definition and Usage

The `popitem()` method removes the item that was last inserted into the dictionary. In versions before 3.7, the `popitem()` method removes a random item.

The removed item is the return value of the `popitem()` method, as a tuple, see example below.

Syntax

`dictionary.popitem()`

Example

The removed item is the return value of the `popitem()` method:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.popitem()  
print(x)
```

setdefault() Method**Example**

Get the value of the "model" item:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.setdefault("model", "Bronco")  
print(x)
```

Definition and Usage

The `setdefault()` method returns the value of the item with the specified key.

If the key does not exist, insert the key, with the specified value, see example below

Syntax

`dictionary.setdefault(keyname, value)`

Parameter Values

Parameter	Description
<i>keyname</i>	Required. The keyname of the item you want to return the value from

<i>value</i>	Optional. If the key exist, this parameter has no effect. If the key does not exist, this value becomes the key's value Default value None
--------------	---

Example

Get the value of the "color" item, if the "color" item does not exist, insert "color" with the value "white":

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
x = car.setdefault("color", "white")
print(x)
```

update() Method**Example**

Insert an item to the dictionary:

```
car = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
car.update({"color": "White"})
print(car)
```

Definition and Usage

The `update()` method inserts the specified items to the dictionary.

The specified items can be a dictionary, or an iterable object with key value pairs.

Syntax

`dictionary.update(iterable)`

Parameter Values

Parameter	Description
<i>iterable</i>	A dictionary or an iterable object with key value pairs, that will be inserted to the dictionary

values() Method**Example**

Return the values:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.values()  
print(x)
```

Definition and Usage

The `values()` method returns a view object. The view object contains the values of the dictionary, as a list.

The view object will reflect any changes done to the dictionary, see example below.

Syntax

`dictionary.values()`

Example

When a values is changed in the dictionary, the view object also gets updated:

```
car = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = car.values()  
car["year"] = 2018  
print(x)
```

13. Advanced Python List Methods and Techniques

Lists are just like dynamically sized arrays, declared in other languages (vector in C++ and ArrayList in Java). Lists need not be homogeneous always which makes it a most powerful tool in Python. A single list may contain DataTypes like Integers, Strings, as well as Objects. Lists are mutable, and hence, they can be altered even after their creation.

Lists are one of the most powerful tools in python. They have a lot of hidden tricks which makes them extremely versatile. Let's explore some of these useful techniques which make our lives much easier !!!

Sorting a list

To sort a list in ascending or descending order, we use the [`sort\(\)`](#) function with the following syntax:

For ascending order:

```
list.sort()
```

For descending order:

```
list.sort(reverse=True)
```

Example:

```
# sorting a list using sort() function
my_list = [5, 2, 90, 24, 10]
# sorting in ascending order permanently
# changes the order of the list
my_list.sort()
print(my_list)
# sorting in descending order permanently
# changes the order of the list
my_list.sort(reverse=True)
print(my_list)
```

Output:

```
[2, 5, 10, 24, 90]
[90, 24, 10, 5, 2]
```

To temporarily change the order of the list use the [sorted\(\)](#) function with the syntax:

list.sorted()

```
# temporary sorting using sorted() method
my_list = [5, 2, 90, 24, 10]
# ascending order
print(sorted(my_list))
# descending order
my_list_2 = sorted(my_list)
print(my_list)
```

Output:

```
[2, 5, 10, 24, 90]
[5, 2, 90, 24, 10]
```

Reversing a list

To reverse the order of a list we use the [reverse\(\)](#) function. Its syntax is:

list.reverse()

Example:

```
# reverse a list using reverse()
my_list = [5, 2, 90, 24, 10]
# reverse
my_list.reverse()
print(my_list)
```

Output:

```
[10, 24, 90, 2, 5]
```

Or we could apply list comprehension to reverse a list:

```
list = list[::-1]
# reverse using list comprehension
my_list = [5, 2, 90, 24, 10]
# reverse
print(my_list[::-1])
```

Output:

```
[10, 24, 90, 2, 5]
```

Removing duplicates

As dictionaries in python do not contain duplicate keys. We use the [*dict.fromkeys\(\)*](#) function to convert our list into a dictionary with list elements as keys. And then we convert the dictionary back to list.

This is a powerful trick to automatically remove all the duplicates. Its syntax is:

```
My_list = ['a', 'b', 'c', 'b', 'a']
Mylist = list(dict.fromkeys(My_List))
```

Example:

```
# removing duplicates from a list using dictionaries
my_list_1 = [5, 2, 90, 24, 10, 2, 90, 34]
my_list_2 = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
# removing duplicates from list 1
my_list_1 = list(dict.fromkeys(my_list_1))
print(my_list_1)
# removing duplicates from list 2
my_list_2 = list(dict.fromkeys(my_list_2))
print(my_list_2)
```

Output:

```
[5, 2, 90, 24, 10, 34]
['a', 'b', 'c', 'd', 'e']
```

Filtering a list

Python lists can be filtered with the help of [*filter\(\)*](#) function or with the help of list comprehension. Below is the syntax:

```
My_list = list(filter(filter_function , iterable_item))
```

The Filter function returns an iterator object which we must convert back into the list.

Example:

```
# filtering with the help of filter() function
# creating a filter function filter all the values less than 20
# filter function
```

```
def my_filter(n):  
    if n > 20:  
        return n  
# driver code  
if __name__ == "__main__":  
    my_list = [5, 2, 90, 24, 10, 2, 95, 36]  
    my_filtered_list = list(filter(my_filter, my_list))  
    print(my_filtered_list)
```

Output:

```
[90, 24, 95, 36]
```

We can also filter using list comprehension. It is a much easier and elegant way to filter lists, below is the syntax:

```
My_list = [item for item in my_list if (condition)]
```

Example:

```
# filtering with the help of list comprehension  
my_list = [5, 2, 90, 24, 10, 2, 95, 36]  
# an elegant way to sort the list  
my_list = [item for item in my_list if item > 20]  
print(my_list)
```

Output:

```
[90, 24, 95, 36]
```

Modifying list

To modify the values in the list with the help of an external function, we make use of the map() function. Map() function returns a map object (iterator) of the results after applying the given function to each element of a given iterable(list, tuple etc.). Below is the syntax:

```
My_list = list(map(function,iterable))
```

Example:

```
# using map() function to modify the text  
def squaring(n):  
    return n**2  
# driver code  
if __name__ == "__main__":  
    my_list = [5, 2, 90, 24, 10, 2, 95, 36]  
    my_squared_list = list(map(squaring, my_list))  
    print(my_squared_list)
```

Output:

```
[25, 4, 8100, 576, 100, 4, 9025, 1296]
```

A much cleaner approach is using list comprehension.

Example:

```
# the same result can be obtained by a much pythonic approach
# i.e., by using list comprehension
my_list = [5, 2, 90, 24, 10, 2, 95, 36]
print([i**2 for i in my_list])
```

Output:

```
[25, 4, 8100, 576, 100, 4, 9025, 1296]
```

Combining lists

We can even combine lists with the help of the [zip\(\)](#) function which results in a list of tuples. Here each item from list A is combined with corresponding elements from list B in the form of a tuple. Below is the syntax:

```
My_list = zip(list_1, list_2)
```

Example:

```
# combining lists with the help of zip() function
my_list_1 = [5, 2, 90, 24, 10]
my_list_2 = [6, 3, 91, 25, 12]
# combined
my_combined_list = list(zip(my_list_1, my_list_2))
print(my_combined_list)
```

Output:

```
[(5, 6), (2, 3), (90, 91), (24, 25), (10, 12)]
```

Finding the most common item

To find the most frequent element we make use of the [set\(\)](#) function. The [set\(\)](#) function removes all the duplicates from the list, and the [max\(\)](#) function returns the most frequent element (which is found with the help of 'key'). The key is an optional single argument function. Below is the syntax:

```
Most_frequent_value = max(set(my_list), key=mylist.count)
```

Example :

```
# to find the most frequent element from the list
my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
most_frequent_value = max(set(my_list), key=my_list.count)
print("The most common element is:", most_frequent_value)
```

Output:

```
The most common element is: a
```

Checking for membership in a list

To check whether an item exists in a list, we use the *in* statement.

Example:

```
my_list = ['a', 'a', 'a', 'b', 'c', 'd', 'd', 'e']
# to check whether 'c' is a member of my_list
```

```
# returns true if present
print('c' in my_list)
# to check whether 'f' is a member of my_list
# returns false if not present
print('f' in my_list)
```

Output:

```
True
False
```

Flatten a list of lists

Sometimes we encounter a list where each element in itself is a list. To convert a list of lists into a single list, we use list comprehension.

```
my_list = [item for List in list_of_lists for item in List ]
```

Example:

```
# to flatten a list_of_lists by using list comprehension
list_of_lists = [[1, 2],
                  [3, 4],
                  [5, 6],
                  [7, 8]]
```

```
# using list comprehension
my_list = [item for List in list_of_lists for item in List]
print(my_list)
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Create a dictionary with list comprehension in Python

Method 1: Using dict() method

Using dict() method we can convert list comprehension to the dictionary. Here we will pass the list_comprehension like a list of tuple values such that the first value act as a key in the dictionary and the second value act as the value in the dictionary.

Syntax: *dict(list_comprehension)*

Example 1: Python program to create a student list comprehension and convert it into a dictionary.

```
# create a list comprehension with student age
data = [('sravan', 23), ('ojaswi', 15),
        ('rohith', 8), ('gnanesh', 4), ('bobby', 20)]
# using dict method
dict(data)
```

Output:

```
{'bobby': 20, 'gnanesh': 4, 'ojaswi': 15, 'rohith': 8, 'sravan': 23}
```

We can also use the following for loop inside the dict() method.

Syntax: *dict([(key,value) for key,value in data])*

Example 2:

```
# create a list comprehension with student age
data = [('sravan', 23), ('ojaswi', 15),
        ('rohith', 8), ('gnanesh', 4), ('bobby', 20)]

# using dict method inside for loop
dict([(key, value) for key, value in data])
```

Output:

```
{'bobby': 20, 'gnanesh': 4, 'ojaswi': 15, 'rohith': 8, 'sravan': 23}
```

Method 2: Using `zip()` with `dict()` method

Here in this method, we will create two lists such that values in the first list will be the keys and the second list values will be the values in the dictionary.

Syntax: *dict(zip(key_list,value_list))*

Example 1:

```
# create a list with student name
name = ['sravan', 'ojaswi', 'rohith', 'gnanesh', 'bobby']
# create a list with student age
age = [23, 21, 32, 11, 23]
# using dict method with zip()
dict(zip(name, age))
```

Output:

```
{'bobby': 23, 'gnanesh': 11, 'ojaswi': 21, 'rohith': 32, 'sravan': 23}
```

Method 3: Using Iterable

Here we can iterate over the given data using iterable for loop.

Syntax: *{key: value for (key, value) in data}*

Example:

```
# create a list comprehension with student age
data = [('sravan', 23), ('ojaswi', 15),
        ('rohith', 8), ('gnanesh', 4), ('bobby', 20)]
# display using iterable method
{key: value for (key, value) in data}
```

Output:

```
{'bobby': 20, 'gnanesh': 4, 'ojaswi': 15, 'rohith': 8, 'sravan': 23}
```

Method 4: Add Filter

We can add a filter to the iterable to a list comprehension to create a dictionary only for particular data based on condition. Filtering means adding values to dictionary-based on conditions.

Syntax: {key: value for (key, value) in data condition}

Example:

```
# create a list comprehension with student age
data = [('sravan', 23), ('ojaswi', 15),
        ('rohith', 8), ('gnanesh', 4), ('bobby', 20)]
# create a dictionary with list
# comprehension if value is equal to 20
print({key: value for (key, value) in data if value == 20})
# create a dictionary with list
# comprehension if value is greater than to 10
print({key: value for (key, value) in data if value > 10})
# create a dictionary with list
# comprehension if key is sravan
print({key: value for (key, value) in data if key == 'sravan'})
```

Output:

```
{'bobby': 20}
{'sravan': 23, 'ojaswi': 15, 'bobby': 20}
{'sravan': 23}
```

Advanced List Processing - List Comprehension

Python supports a concept called "list comprehensions". It can be used to construct lists in a very natural, easy way, like a mathematician is used to do.

Advanced List Processing - List Comprehension

- Python supports a concept called "list comprehensions". It can be used to construct lists in a very natural, easy way, like a mathematician is used to do.
- In Python, we can write the expression almost exactly like a mathematics with special cryptic syntax.

For example:

```
>>> L = (x**2 for x in range(10))
>>> L
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>>
```

- The above code creates a list of square numbers from 0 to 10. This list can be created using a simplified expression "x**2 for x in range(10)". This is basically list comprehension.
- List comprehensions apply an arbitrary expression to items in an iteration.
- The list comprehension is achieved using the tools like map, lambda and filter.

1. Map

- The map() function applies the values to every member of the list and returns the result.
- The map function takes two arguments –

result = map(function, sequence)

For example

```
>>> def increment_by_three(x):
return x+3
>>> new_list=list(map(increment_by_three,range(10)))
>>> new_list
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
>>>
```

Another Example :

There is a built in function for calculating length of the string and i.e. len(). We can pass this function to map and find the length of every element present in the list as a list.

```
>>> names=['Sandip', 'Lucky','Mahesh','Sachin']
>>> lengths=list(map(len, names))
>>> lengths
[6, 5, 6, 6]
>>>
```

2. Filter

- The Filter() function selects some of the elements and filters out others. If we use
- if with list comprehension, it is almost equivalent to the filter built-in.
- This function takes two arguments –
- result = filter(function, sequence)

For example –

```
>>> def even_fun(x):
return x%2== 0
>>> r = filter(even_fun,(1,2,3,4,5))
>>> list(r) [2, 4]
>>>
```

- In above example, using **filter()** function the odd numbers are filtered out and only even numbers are displayed.
- filter() works in the similar manner like map(), it applies function to all the elements of the list.

3. Lambda

- The lambda function is a simple inline function that is used to evaluate the expression using the given list of arguments.

- The syntax is

lambda argument_list : expression

where the **argument_list** consists of a comma separated list of arguments and the expression is an arithmetic expression using these arguments.

For example:

```
>>> sum = lambda x,y:x+y
>>> sum(1,3)
4
>>>
```

Thus the expression $x+y$ is evaluated and the result of evaluation is returned.

4. Reduce

- The kind of function that combines sequence of elements into a single value is called reduce function.
- The reduce function also takes two arguments -
Result = reduce(function, sequence)
- The function **reduce()** had been dropped from the core of Python when migrating to Python 3. Hence for using the reduce function we have to import functools to be capable of using reduce.

For example

```
>>> import functools
>>> functools.reduce(lambda x,y:x+y,[1,2,3,4])
10
>>>
```

Example 4.4.1 *Following is a list of items purchased*

Write a python program, which returns a list with 2 tuples - Each tuple consists of item no, and product of price and quantity.

If value of the order is less than 100 then the product is increased by 10. Write a program in Python using map and lambda functions

Solution :

```
>>> order=[["111", 12,20],["112",5,100],["113",7,5]]
>>> min_order=100
>>> Total_Bill=list(map(lambda x:x if x[1]>=min_order else(x[0],x[1]+10),
map(lambda x:(x[0],x[1]*x[2]),order)))
```

```
>>> print(Total_Bill)
[('111', 240), ('112', 500), ('113', 45)]
>>>
```

Example 4.4.2 Write a Python program to display power of 2 using Anonymous function.

Solution :

The Anonymous function is a lambda function. We can write the program to display the power of 2 using Anonymous function

Test.py

Python Dictionary Comprehension

Python Dictionary Comprehension Example

Here we have two [lists](#) named keys and value and we are iterating over them with the help of [zip\(\)](#) function.

```
# Python code to demonstrate dictionary
# comprehension
# Lists to represent keys and values
keys = ['a','b','c','d','e']
values = [1,2,3,4,5]
# but this line shows dict comprehension here
myDict = { k:v for (k,v) in zip(keys, values)}
# We can use below too
# myDict = dict(zip(keys, values))
```

```
print (myDict)
```

Output :

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

Using fromkeys() Method

Here we are using the [fromkeys\(\)](#) method that returns a dictionary with specific keys and values.

```
dic=dict.fromkeys(range(5), True)
print(dic)
```

Output:

```
{0: True, 1: True, 2: True, 3: True, 4: True}
```

Using dictionary comprehension make dictionary

Example 1:

```
# Python code to demonstrate dictionary
# creation using list comprehension
myDict = {x: x**2 for x in [1,2,3,4,5]}
print (myDict)
```

Output :

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Example 2:

```
sDict = {x.upper(): x*3 for x in 'coding '}
print (sDict)
```

Output :

```
{'O': 'ooo', 'N': 'nnn', 'I': 'iii', 'C': 'ccc', 'D': 'ddd', 'G': 'ggg'}
```

Using conditional statements in dictionary comprehension

Example 1:

We can use Dictionary comprehensions with if and else statements and with other expressions too. This example below maps the numbers to their cubes that are divisible by 4.

```
# Python code to demonstrate dictionary
# comprehension using if.
newdict = {x: x**3 for x in range(10) if x**3 % 4 == 0}
print(newdict)
```

Output :

```
{0: 0, 8: 512, 2: 8, 4: 64, 6: 216}
```

Using nested dictionary comprehension

Here we are trying to create a nested [dictionary](#) with the help of dictionary comprehension.

```
# given string
```

```
l="GFG"
# using dictionary comprehension
dic = {
    x: {y: x + y for y in l} for x in l
}
print(dic)
```

Output:

```
{'G': {'G': 'GG', 'F': 'GF'}, 'F': {'G': 'FG', 'F': 'FF'}}
```

14.SETS: CREATING A SET

Python set is an unordered collection of multiple items having different datatypes. In [Python](#), sets are mutable, unindexed and do not contain duplicates. The order of elements in a set is not preserved and can change.

Creating a Set in Python

In Python, the most basic and efficient method for creating a set is using curly braces.

Example:

```
set1 = {1, 2, 3, 4}
print(set1)
```

Output

```
{1, 2, 3, 4}
```

Using the set() function

Python Sets can be created by using the built-in [set\(\) function](#) with an iterable object or a sequence by placing the sequence inside curly braces, separated by a 'comma'.

Note: A Python set cannot have mutable elements like a list or dictionary, as it is immutable.

Example:

```
# Creating a Set
set1 = set()
print(set1)
set1 = set("GeeksForGeeks")
print(set1)
# Creating a Set with the use of a List
set1 = set(["Geeks", "For", "Geeks"])
print(set1)
# Creating a Set with the use of a tuple
tup = ("Geeks", "for", "Geeks")
print(set(tup))
# Creating a Set with the use of a dictionary
d = {"Geeks": 1, "for": 2, "Geeks": 3}
```

```
print(set(d))
```

Output

```
set()
{'e', 'r', 'o', 'k', 'G', 's', 'F'}
{'For', 'Geeks'}
{'for', 'Geeks'}
{'for', 'Geeks'}
```

Time complexity: $O(n)$, where n is the length of the input string, list, tuple or dictionary.

Auxiliary space: $O(n)$, where n is the length of the input string, list, tuple or dictionary.

Unordered, Unindexed and Mutability

In set, the order of elements is not guaranteed to be the same as the order in which they were added. The output could vary each time we run the program. Also the duplicate items entered are removed by itself.

Sets do not support indexing. Trying to access an element by index (`set[0]`) raises a [TypeError](#).

We can add elements to the set using `add()`. We can remove elements from the set using `remove()`. The set changes after these operations, demonstrating its mutability. However, we cannot change its items directly.

Example:

```
# Creating a set
set1 = {3, 1, 4, 1, 5, 9, 2}
# Unordered: The order of elements is not preserved
print(set1) # Output may vary: {1, 2, 3, 4, 5, 9}
# Unindexed: Accessing elements by index is not possible
# This will raise a TypeError
try:
    print(set1[0])
except TypeError as e:
    print(e)
```

Output

```
{1, 2, 3, 4, 5, 9}
'set' object is not subscriptable
```

15.ADDING ELEMENTS TO A SET IN PYTHON

We can add items to a set using [add\(\) method](#) and [update\(\) method](#). `add()` method can be used to add only a single item. To add multiple items we use `update()` method.

Example:

```
# Creating a set
set1 = {1, 2, 3}
# Add one item
```

```
set1.add(4)
# Add multiple items
set1.update([5, 6])
print(set1)
```

Output

```
{1, 2, 3, 4, 5, 6}
```

Accessing a Set in Python

We can loop through a set to access set items as set is unindexed and do not support accessing elements by indexing. Also we can use [in keyword](#) which is membership operator to check if an item exists in a set.

Example:

```
set1 = set(["Geeks", "For", "Geeks."])
# Accessing element using For loop
for i in set1:
    print(i, end=" ")
# Checking the element# using in keyword
print("Geeks" in set1)
```

Output

```
Geeks For Geeks. True
```

Explanation:

- o This loop will print each item in the set. Since sets are unordered, the order of items printed is not guaranteed.
- o This code checks if the number 4 is in set1 and prints a corresponding message.

16. REMOVING ELEMENTS FROM THE SET IN PYTHON

We can remove an element from a set in Python using several methods: `remove()`, `discard()` and `pop()`. Each method works slightly differently :

- Using [remove\(\) Method](#) or [discard\(\) Method](#)
- Using [pop\(\) Method](#)
- Using [clear\(\) Method](#)

Using remove() Method or discard() Method

`remove()` method removes a specified element from the set. If the element is not present in the set, it raises a `KeyError`. `discard()` method also removes a specified element from the set. Unlike `remove()`, if the element is not found, it does not raise an error.

```
# Using Remove Method
set1 = {1, 2, 3, 4, 5}
```

```

set1.remove(3)
print(set1)
# Attempting to remove an element that does not exist
try:
    set1.remove(10)
except KeyError as e:
    print("Error:", e)

# Using discard() Method
set1.discard(4)
print(set1)
# Attempting to discard an element that does not exist
set1.discard(10) # No error raised
print(set1)

```

Output

```

{1, 2, 4, 5}
Error: 10
{1, 2, 5}
{1, 2, 5}

```

Using pop() Method

pop() method removes and returns an arbitrary element from the set. This means we don't know which element will be removed. If the set is empty, it raises a KeyError.

Note: *If the set is unordered then there's no such way to determine which element is popped by using the pop() function.*

```

set1 = {1, 2, 3, 4, 5}
val = set1.pop()
print(val)
print(set1)
# Using pop on an empty set
set1.clear() # Clear the set to make it empty
try:
    set1.pop()
except KeyError as e:
    print("Error:", e)

```

Output

```

1
{2, 3, 4, 5}
Error: 'pop from an empty set'

```

Using clear() Method

clear() method removes all elements from the set, leaving it empty.

```

set1 = {1, 2, 3, 4, 5}
set1.clear()
print(set1)

```

Output

```
set()
```

Frozen Sets in Python

A [frozenset](#) in Python is a built-in data type that is similar to a set but with one key difference that is immutability. This means that once a frozenset is created, we cannot modify its elements that is we cannot add, remove or change any items in it. Like regular sets, a frozenset cannot contain duplicate elements.

If no parameters are passed, it returns an empty frozenset.

```
# Creating a frozenset from a list
fset = frozenset([1, 2, 3, 4, 5])
print(fset)
# Creating a frozenset from a set
set1 = {3, 1, 4, 1, 5}
fset = frozenset(set1)
print(fset)
```

Output

```
frozenset({1, 2, 3, 4, 5})
frozenset({1, 3, 4, 5})
```

Typecasting Objects into Sets

Typecasting objects into sets in Python refers to converting various data types into a set. Python provides the set() constructor to perform this typecasting, allowing us to convert lists, tuples and strings into sets.

```
# Typecasting list into set
li = [1, 2, 3, 3, 4, 5, 5, 6, 2]
set1 = set(li)
print(set1)
# Typecasting string into set
s = "GeeksforGeeks"
set1 = set(s)
print(set1)
# Typecasting dictionary into set
d = {1: "One", 2: "Two", 3: "Three"}
set1 = set(d)
print(set1)
```

Output

```
{1, 2, 3, 4, 5, 6}
{'f', 'G', 's', 'k', 'r', 'e', 'o'}
{1, 2, 3}
```

Advantages of Set in Python

- o **Unique Elements:** Sets can only contain unique elements, so they can be useful for removing duplicates from a collection of data.
- o **Fast Membership Testing:** Sets are optimized for fast membership testing, so they can be useful for determining whether a value is in a collection or not.
- o **Mathematical Set Operations:** Sets support mathematical set operations like union, intersection and difference, which can be useful for working with sets of data.
- o **Mutable:** Sets are mutable, which means that you can add or remove elements from a set after it has been created.

Disadvantages of Sets in Python

- **Unordered:** Sets are unordered, which means that you cannot rely on the order of the data in the set. This can make it difficult to access or process data in a specific order.
- **Limited Functionality:** Sets have limited functionality compared to lists, as they do not support methods like `append()` or `pop()`. This can make it more difficult to modify or manipulate data stored in a set.
- **Memory Usage:** Sets can consume more memory than lists, especially for small datasets. This is because each element in a set requires additional memory to store a hash value.
- **Less Commonly Used:** Sets are less commonly used than lists and dictionaries in Python, which means that there may be fewer resources or libraries available for working with them. This can make it more difficult to find solutions to problems or to get help with debugging.

17. OTHER SET OPERATIONS IN PYTHON

Function	Description
<code>add()</code>	Adds an element to a set
<code>remove()</code>	Removes an element from a set. If the element is not present in the set, raise a <code>KeyError</code>

Function	Description
<u>clear()</u>	Removes all elements form a set
<u>copy()</u>	Returns a shallow copy of a set
<u>pop()</u>	Removes and returns an arbitrary set element. Raise KeyError if the set is empty
<u>update()</u>	Updates a set with the union of itself and others
<u>union()</u>	Returns the union of sets in a new set
<u>difference()</u>	Returns the difference of two or more sets as a new set
<u>difference_update()</u>	Removes all elements of another set from this set
<u>discard()</u>	Removes an element from set if it is a member. (Do nothing if the element is not in set)
<u>intersection()</u>	Returns the intersection of two sets as a new set
<u>intersection_update()</u>	Updates the set with the intersection of itself and another
<u>isdisjoint()</u>	Returns True if two sets have a null intersection

Function	Description
<u>issubset()</u>	Returns True if another set contains this set
<u>issuperset()</u>	Returns True if this set contains another set
<u>symmetric_difference()</u>	Returns the symmetric difference of two sets as a new set
<u>symmetric_difference_update()</u>	Updates a set with the symmetric difference of itself and another